UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Achmad Faiz Yudha Ramadhan 2428240113
Kelas	SI5C
Judul project / topik API	Movies RESTful API — Postman Collection dan Automated Test Script
Tautan repositori	https://github.com/FaizZzhue/tugas1-restful-2428240113
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	19 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `/movies`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

API yang diuji adalah Movies RESTful API untuk mengelola data film, meliputi melihat daftar film, mencari film berdasarkan ID, memfilter genre, menambah, mengubah, dan menghapus film. API dibuat menggunakan Express.js dan diuji melalui Postman Collection dengan Environment untuk base_url, api_key, dan movieId.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/movies	Tidak	200	Mengambil seluruh data film
2	GET	/movies/:id	Tidak	200	Mengambil detail film berdasarkan ID
3	GET	/movies?genre=Drama	Tidak	200	Memfilter film berdasarkan genre
4	POST	/movies	Ya	201	Menambahkan film baru
5	PUT	/movies/:id	Ya	200	Memperbarui data film
6	DELETE	/movies/:id	Ya	200	Menghapus film
7	POST	/movies	Ya	401	Skenario negatif: tanpa API key
8	POST	/movies	Ya	400	Skenario negatif: body tidak lengkap
9	GET	/movies/99999	Tidak	404	Skenario negatif: ID film tidak ditemukan

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server
2	api_key	nilai lokal (jangan dipublikasikan)	Nilai header kunci API

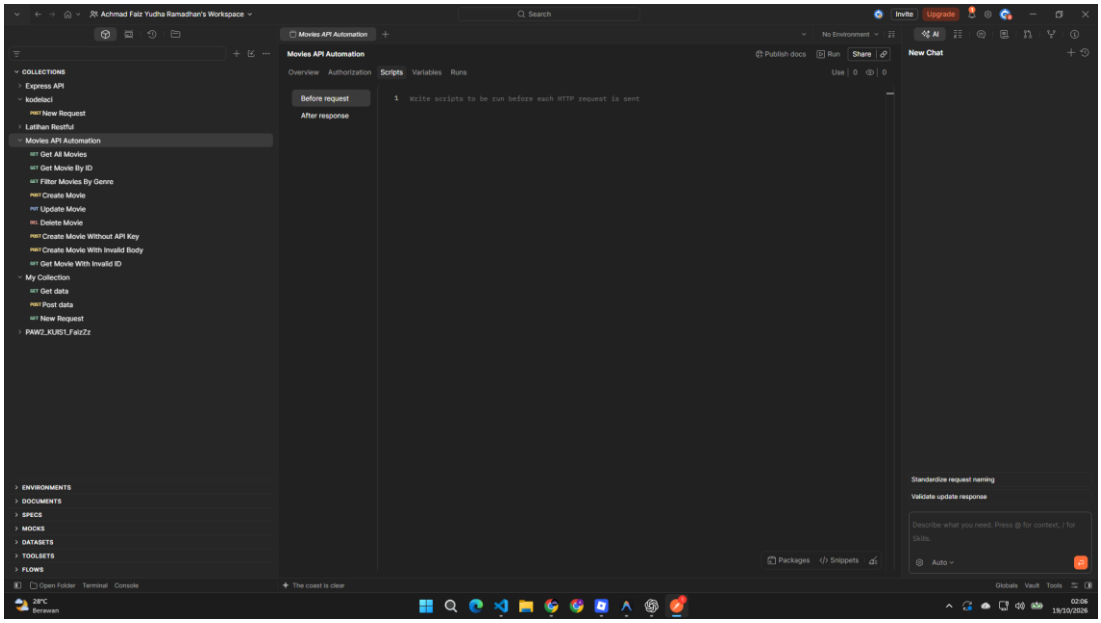
No	Nama variabel	Nilai (contoh)	Fungsi
3	movieId	diisi dari respons POST	Menyimpan ID film yang baru dibuat untuk request PUT/DELETE
4	—	—	Tidak ada variabel lain yang teridentifikasi dari screenshot

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Collection bernama “Movies API Automation” mengelompokkan request untuk operasi GET, filter, POST, PUT, DELETE, serta tiga skenario negatif. Request disusun dari pengambilan data dan operasi CRUD terlebih dahulu, lalu pengujian kondisi gagal agar alur pengujian mudah diikuti.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
Get All Movies	GET	{{base_url}}/movies	Movies API Automation	
Get Movie By ID	GET	{{base_url}}/movies/{{movieId}}	Movies API Automation	
Filter Movies By Genre	GET	{{base_url}}/movies?genre=Drama	Movies API Automation	

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
Create Movie	POST	{{base_url}}/movies	Movies API Automation	
Update Movie	PUT	{{base_url}}/movies/{{movieId}}	Movies API Automation	
Delete Movie	DELETE	{{base_url}}/movies/{{movieId}}	Movies API Automation	
Create Movie Without API Key	POST	{{base_url}}/movies	Movies API Automation	
Create Movie With Invalid Body	POST	{{base_url}}/movies	Movies API Automation	
Get Movie With Invalid ID	GET	{{base_url}}/movies/99999	Movies API Automation	
—	—	—	—	

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
Get All Movies	200 OK	Respons berupa array; setiap item memiliki id dan judul	
Get Movie By ID	200 OK	Data memiliki id, judul, genre, durasiMenit	
Filter Movies By Genre	200 OK	Semua hasil memiliki genre Drama	
Create Movie	201 Created	status success dan data.id tersedia; simpan movieId	
Update Movie	200 OK	data.id sama dengan movieId dan judul berubah	
Delete Movie	200 OK	status success dan pesan data berhasil dihapus	
Create Movie Without API Key	401 Unauthorized	status error dan pesan API key wajib disertakan	
Create Movie With Invalid Body	400 Bad Request	status error dan pesan field wajib diisi	
Get Movie With Invalid ID	404 Not Found	status error dan pesan ID tidak ditemukan	
—	—	—	

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
// Get All Movies — Scripts > After response
pm.test("Status code harus 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Respons berupa array film", function () {
    const data = pm.response.json();
    pm.expect(data).to.be.an("array");
    if (data.length > 0) {
        pm.expect(data[0]).to.have.property("id");
        pm.expect(data[0]).to.have.property("judul");
    }
});
```

Assertion pertama memastikan endpoint GET /movies mengembalikan HTTP 200. Assertion kedua memeriksa bentuk respons berupa array dan, bila terdapat data, memastikan elemen film mempunyai properti id dan judul sehingga struktur respons yang diterima sesuai kebutuhan.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
// Create Movie — Scripts > After response
pm.test("Status code harus 201", function () {
    pm.response.to.have.status(201);
});

pm.test("Respons berisi data film dan ID", function () {
    const jsonData = pm.response.json();
    pm.expect(jsonData.status).to.eql("success");
    pm.expect(jsonData.data).to.have.property("id");
    pm.environment.set("movieId", jsonData.data.id);
});
```

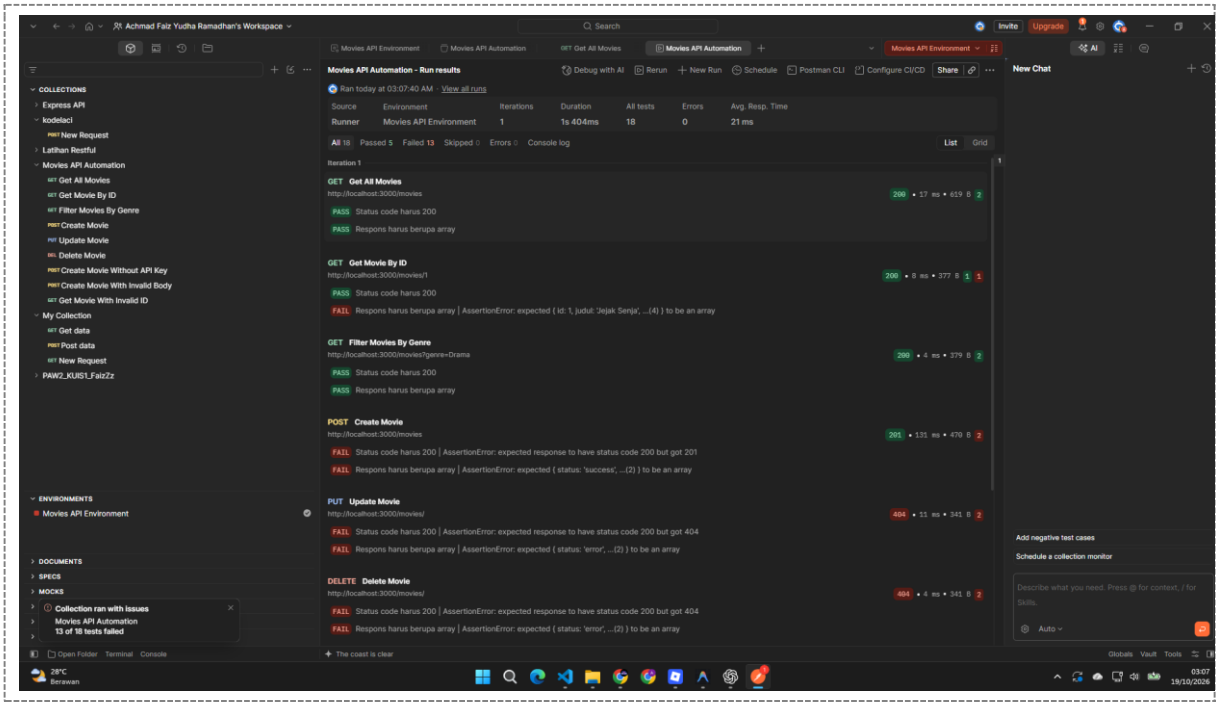
Request POST membuat film “Film Automation Test” dan menyimpan data.id dari respons ke variabel Environment movieId. Request PUT dan DELETE menggunakan {{movieId}} agar mengarah ke film yang sama; karena itu POST perlu dijalankan sebelum PUT dan DELETE, dan urutan request tidak boleh ditukar tanpa mengatur ulang data.

Bab 4 Hasil Collection Runner

Bab ini mendokumentasikan hasil Collection Runner. Pada pengujian awal yang ditampilkan, belum seluruh assertion lulus; hasil ini digunakan untuk mengidentifikasi bagian test script yang perlu diperbaiki.

4.1 Ringkasan Collection Runner

Screenshot menampilkan nama Collection, Environment, 1 iterasi, 18 assertion, 5 test lulus, 13 test gagal, 0 skipped, dan 0 errors. Hasil ini merupakan pengujian awal dan belum memenuhi kriteria seluruh test lulus.



Gambar 2. Hasil awal Collection Runner — 18 assertion, 5 passed, 13 failed.

Hasil Runner memperlihatkan sebagian test berhasil, tetapi masih ada kegagalan. GET by ID mengembalikan objek tunggal sementara assertion mengharapkan array; POST mengembalikan 201 Created sementara assertion mengharapkan 200; PUT dan DELETE membentuk URL /movies/ tanpa ID sehingga menerima 404. Perbaikan perlu dilakukan sebelum menjalankan Runner kembali.

4.2 Rekap angka hasil

Nama Collection	Movies API Automation
Environment yang dipakai	Movies API Environment
Jumlah request dijalankan	1 iterasi; 18 assertion
Jumlah test lulus	5
Jumlah test gagal	13
Tanggal dan jam pengujian	19 Oktober 2026, sekitar 03.07 WIB (waktu yang terlihat pada screenshot Runner)

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Create Movie With Invalid Body	400 Bad Request; field wajib diisi	Request sengaja mengirim body yang hanya berisi judul dan genre	Diharapkan API menolak body yang tidak lengkap; status 400 sesuai skenario
2	Create Movie Without API Key	401 Unauthorized; API key wajib disertakan	Header x-api-key tidak disertakan	API menolak request tanpa kunci API

Bab 5 Newman di Terminal (Pengayaan)

> Bagian Newman tidak dikerjakan. Tidak ada screenshot atau bukti bahwa Collection dijalankan melalui Newman di terminal.

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

Newman tidak dijalankan pada dokumentasi ini, sehingga tidak ada perintah newman run maupun exit code yang dapat dilaporkan. Bagian ini bersifat pengayaan sesuai template.

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/FaizZzhue/tugas1-restful-2428240113
Lokasi berkas Collection (.json)	Movies API Automation.postman_collection.json (tersedia di workspace; pastikan file ini ikut di-push ke repository)
Lokasi berkas Environment (.json)	Movies API Environment (terlihat di Postman; file JSON export belum tersedia pada bukti yang diterima)
Berkas Environment tidak memuat kunci rahasia sungguhan	[ ] Ya [x] Tidak terverifikasi
Repositori dapat dibuka dosen	[ ] Ya [x] Belum diverifikasi langsung

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Assertion yang paling berguna adalah pemeriksaan status code dan struktur JSON, karena keduanya menunjukkan apakah endpoint memberi hasil yang diharapkan sekaligus apakah bentuk datanya benar. Contohnya, skenario tanpa API key harus menghasilkan 401, sedangkan ID film yang tidak ada harus menghasilkan 404.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Pengujian manual memerlukan pengiriman request satu per satu dan pemeriksaan respons secara langsung. Collection Runner dapat mengulang seluruh request dan assertion secara berurutan, tetapi bagian yang perlu diperhatikan adalah urutan POST, PUT, dan DELETE serta penggunaan variabel movieId agar request lanjutan mengarah ke data yang benar.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	Ya
2	Setiap endpoint API saya ada di Collection	Ya
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	Ya
4	Setiap request punya assertion status code dan struktur JSON	Perlu verifikasi semua script
5	Nama setiap test berupa kalimat yang menyatakan harapan	Ada pada contoh script di laporan; pastikan ditempel ke Postman
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	Ya, movieId digunakan oleh PUT/DELETE
7	Server dinyalakan ulang sebelum Runner dijalankan	Belum ada bukti
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	Belum ada screenshot Runner
9	Berkas Environment tidak memuat kunci rahasia sungguhan	Belum terverifikasi; jangan ekspor nilai rahasia
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	Ya setelah PDF final diperiksa

Lampiran B Kriteria Penilaian

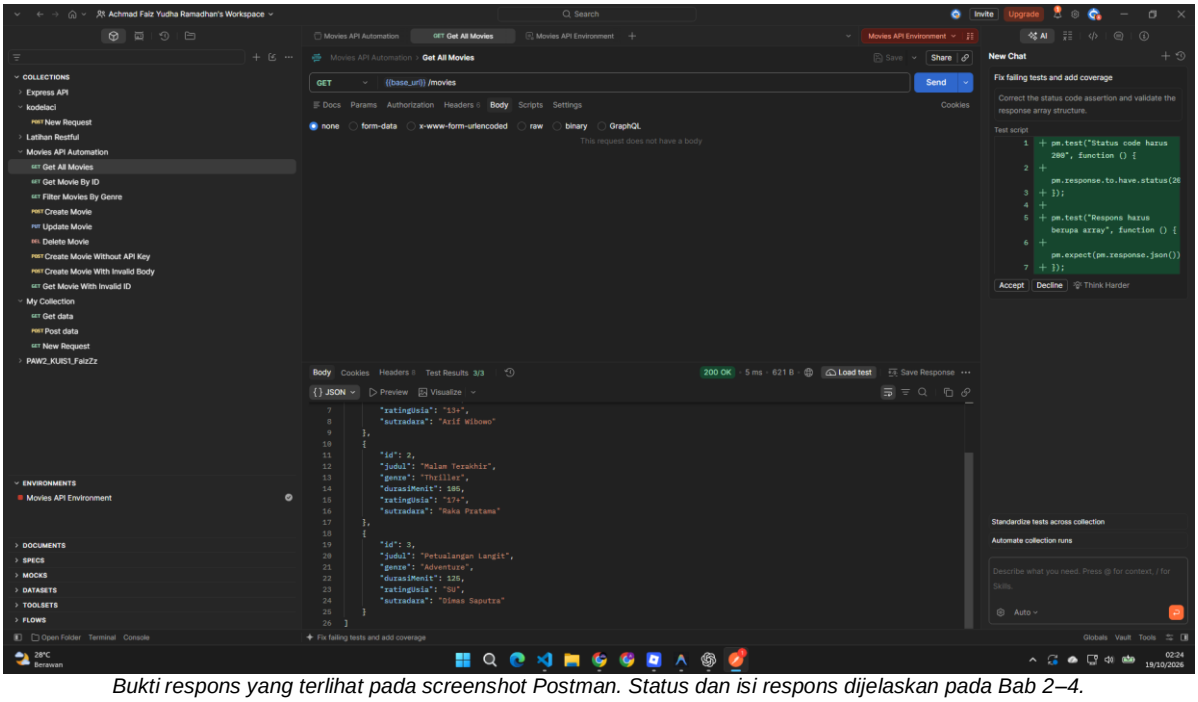
Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Belum lulus: 5 passed, 13 failed; perlu perbaikan
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen

Lampiran C Bukti Pengujian Request di Postman

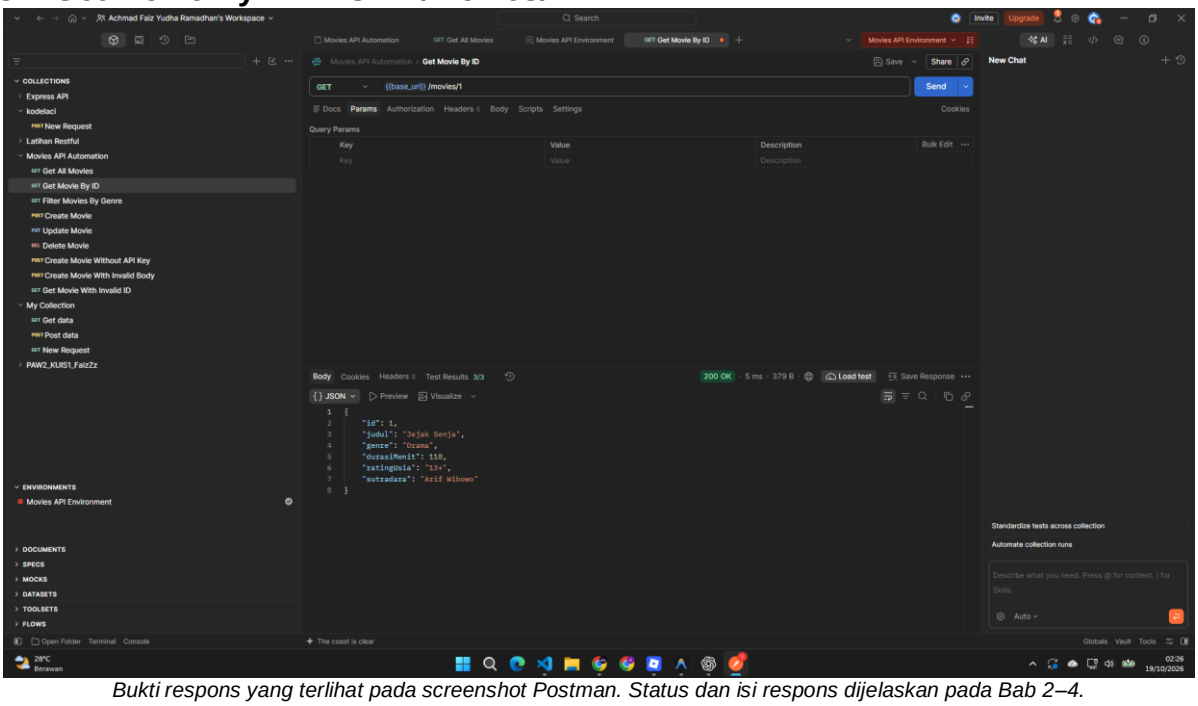
Lampiran ini memuat tangkapan layar request individual. Bukti ringkasan Collection Runner disajikan pada Bab 4. Hasil Runner yang terdokumentasi masih menunjukkan assertion gagal dan perlu diperbaiki sebelum dinyatakan lulus.

C.1 Get All Movies — GET /movies



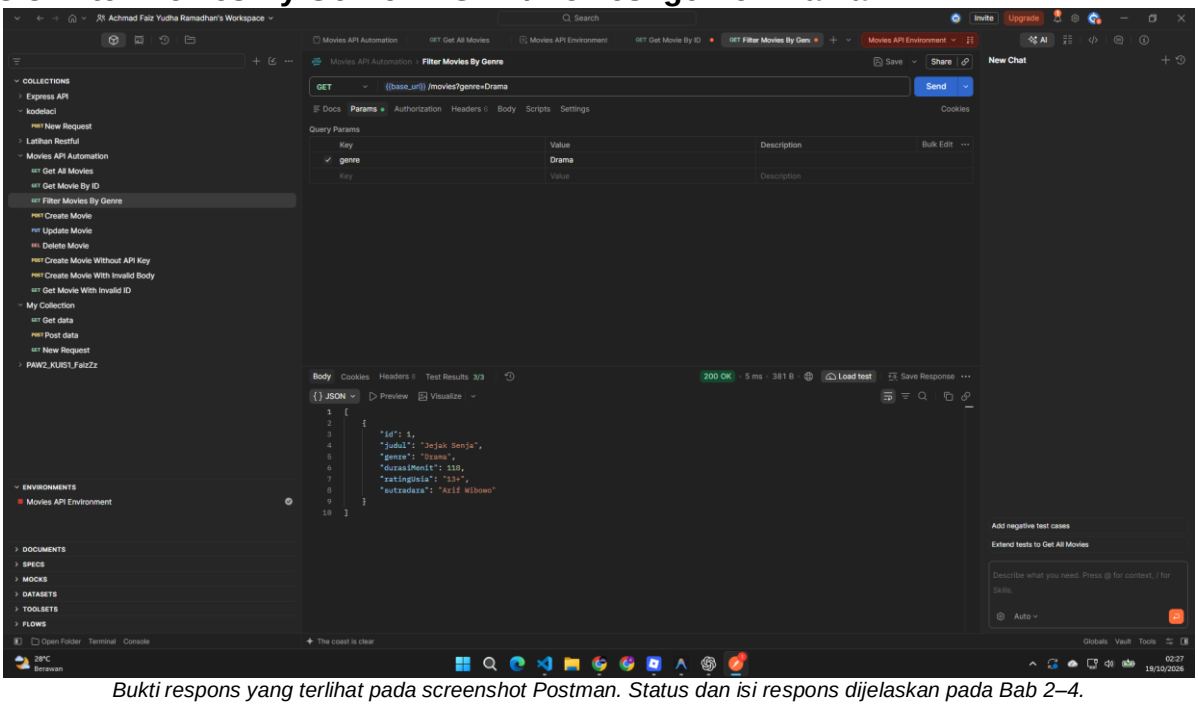
Bukti respons yang terlihat pada screenshot Postman. Status dan isi respons dijelaskan pada Bab 2–4.

C.2 Get Movie By ID — GET /movies/1



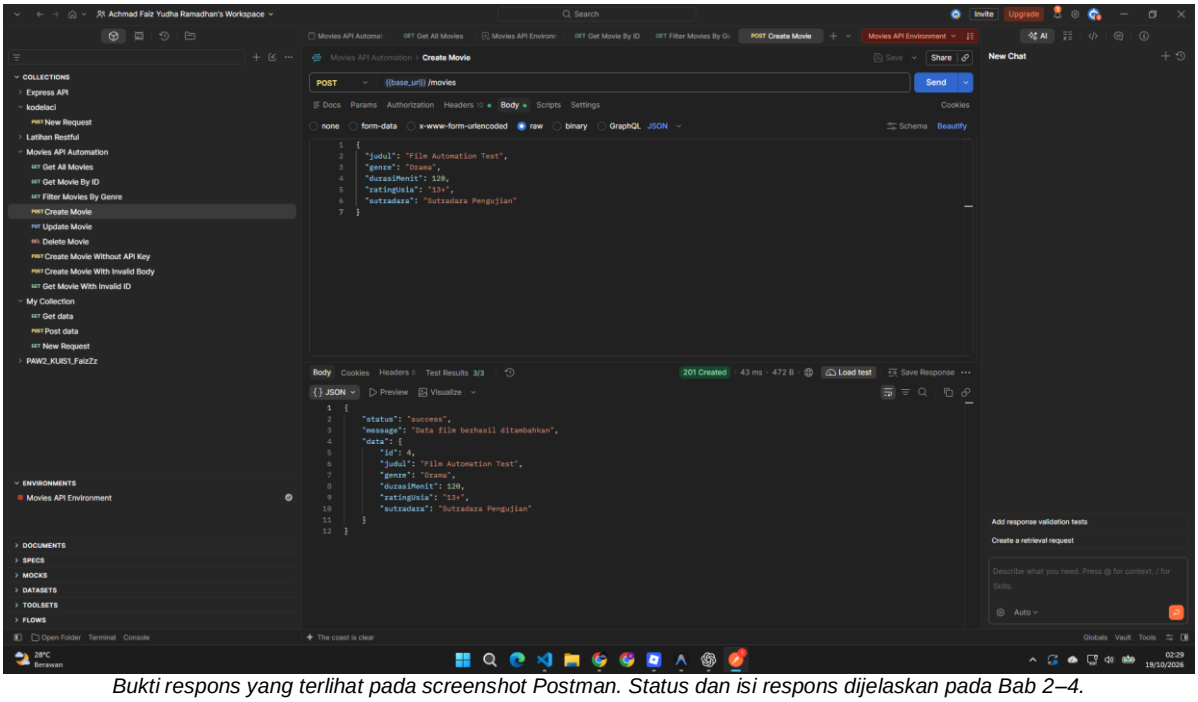
Bukti respons yang terlihat pada screenshot Postman. Status dan isi respons dijelaskan pada Bab 2–4.

C.3 Filter Movies By Genre — GET /movies?genre=Drama



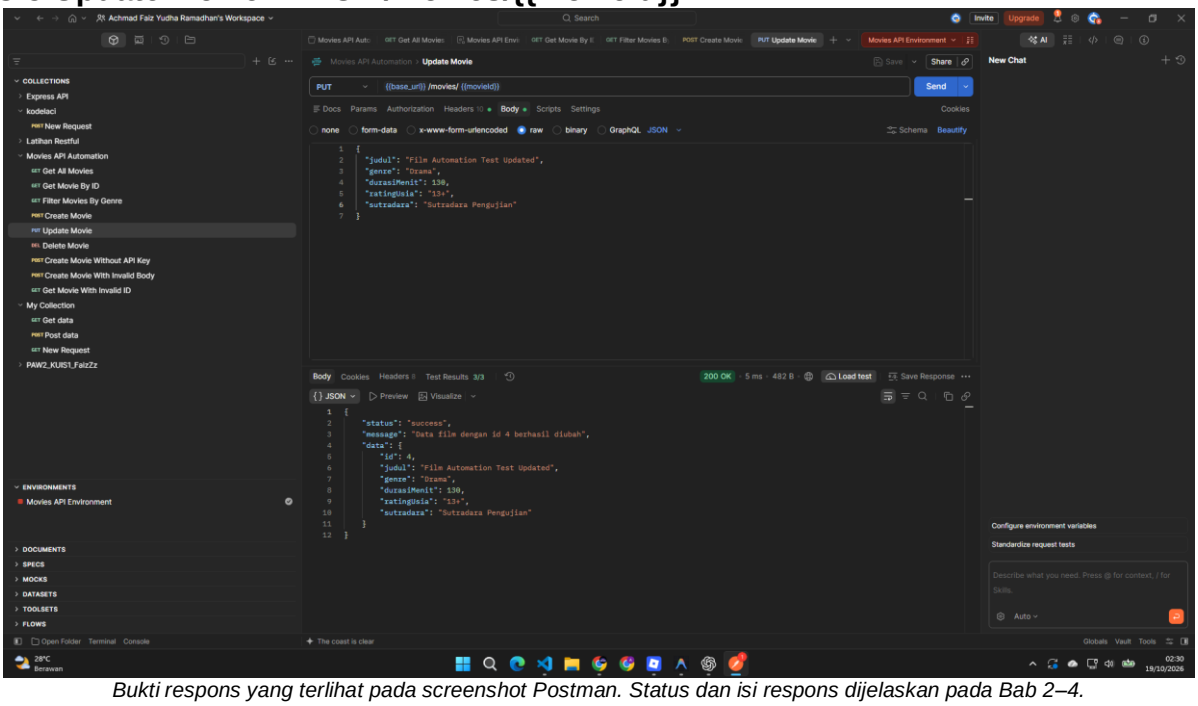
Bukti respons yang terlihat pada screenshot Postman. Status dan isi respons dijelaskan pada Bab 2–4.

C.4 Create Movie — POST /movies



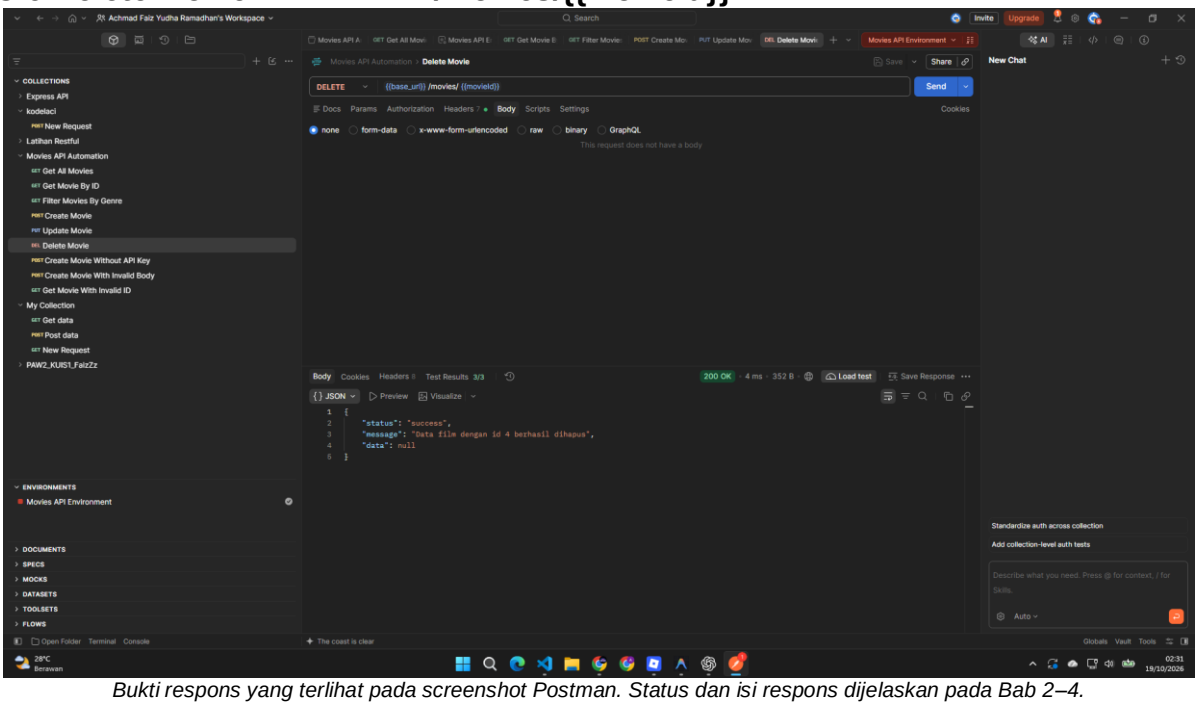
Bukti respons yang terlihat pada screenshot Postman. Status dan isi respons dijelaskan pada Bab 2–4.

C.5 Update Movie — PUT /movies/{{movieId}}



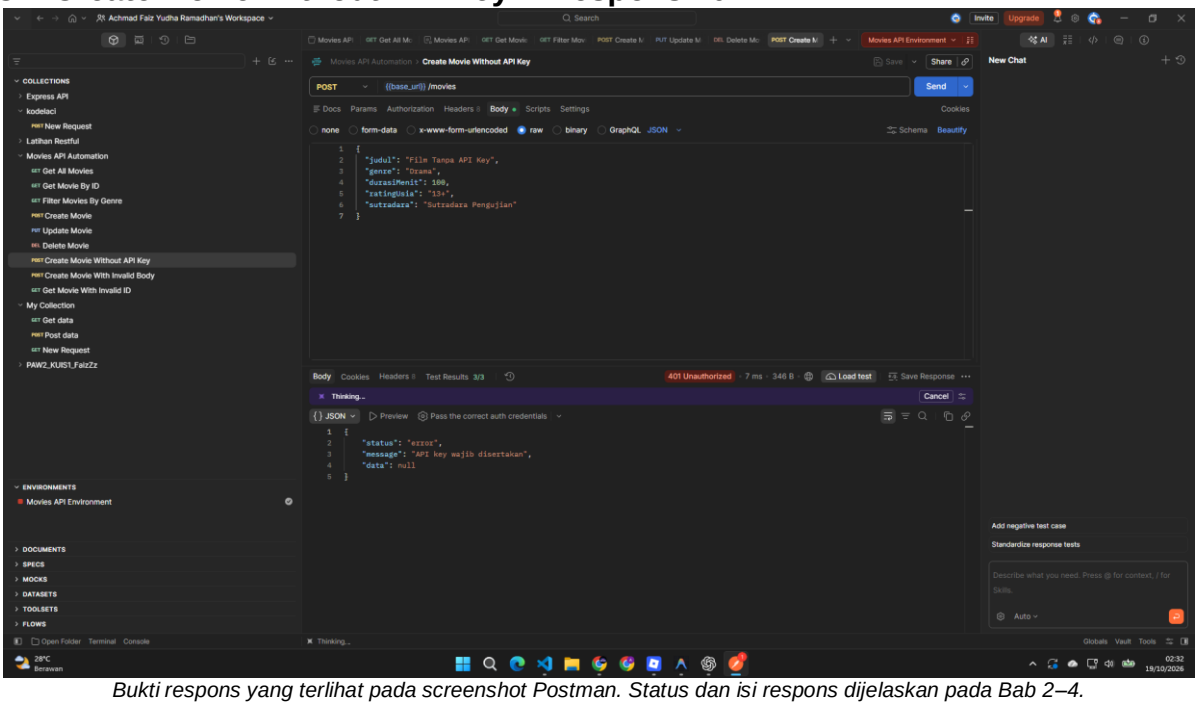
Bukti respons yang terlihat pada screenshot Postman. Status dan isi respons dijelaskan pada Bab 2–4.

C.6 Delete Movie — DELETE /movies/{{movieId}}



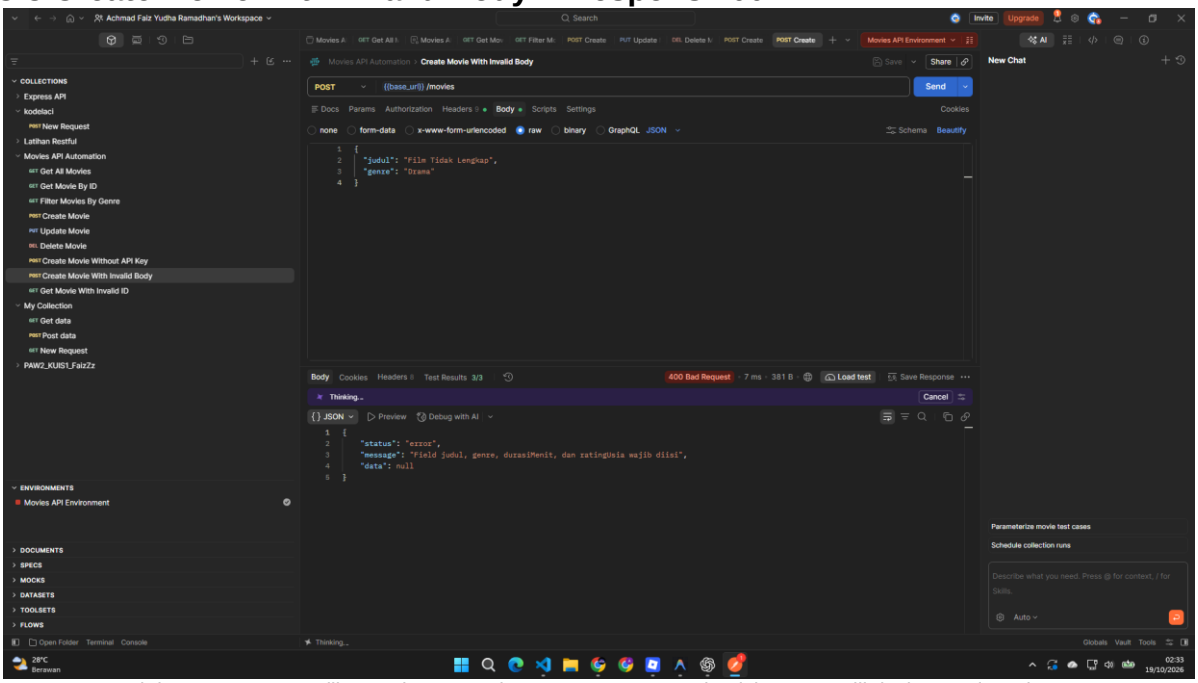
Bukti respons yang terlihat pada screenshot Postman. Status dan isi respons dijelaskan pada Bab 2–4.

C.7 Create Movie Without API Key — respons 401



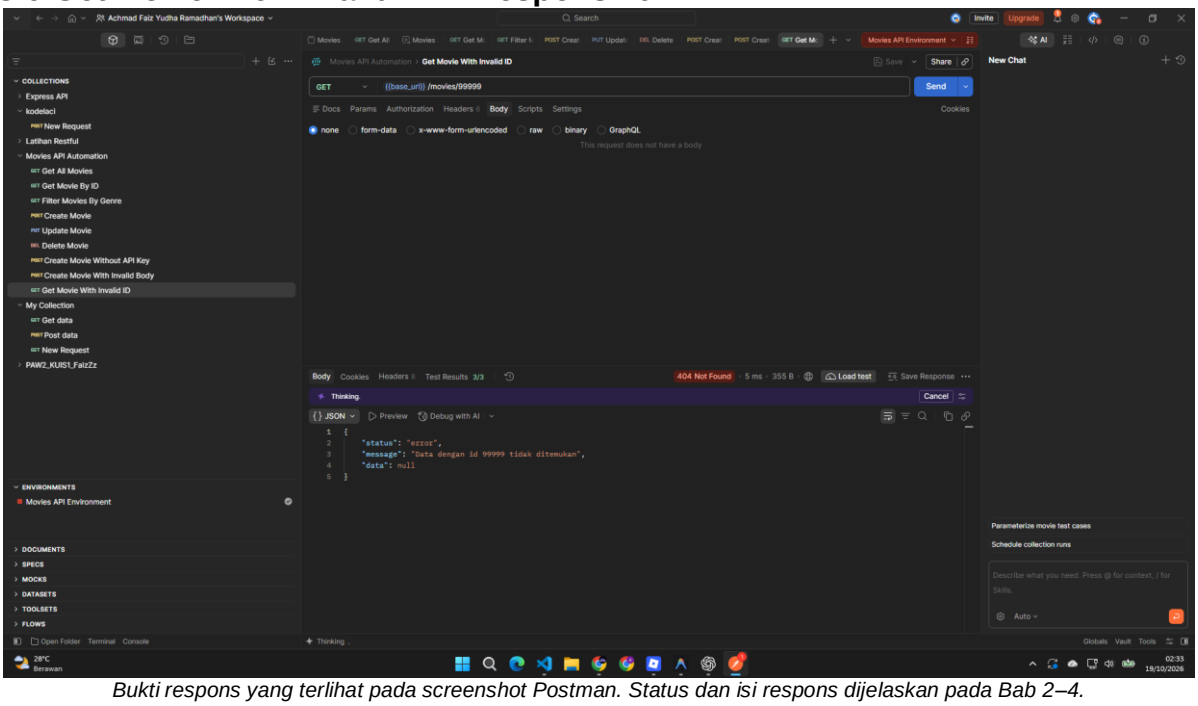
Bukti respons yang terlihat pada screenshot Postman. Status dan isi respons dijelaskan pada Bab 2–4.

C.8 Create Movie With Invalid Body — respons 400



Bukti respons yang terlihat pada screenshot Postman. Status dan isi respons dijelaskan pada Bab 2–4.

C.9 Get Movie With Invalid ID — respons 404



Bukti respons yang terlihat pada screenshot Postman. Status dan isi respons dijelaskan pada Bab 2–4.